SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

# AI-Powered Vulnerability Detection and Secure Code Review Platform

## CSA5802 - DEVSECOPS ENGINEERING AND APPLICATION SECURITY


**TEAM MEMBERS**

Krithika M ( 192521385)

Prathiksha SN (192565090)

Shreya V (192565018)

Sri Janani B( 192421036)

ENGINEER TO EXCEL

# INTRODUCTION

- Modern software applications are increasingly vulnerable to cyber threats due to insecure coding practices and rapidly evolving attack techniques.
- Manual security reviews are time-consuming, error-prone, and difficult to perform consistently for large software projects.
- DevDeployShield is an AI-powered platform designed to automate vulnerability detection and secure code review before deployment.
- The platform enables developers to upload project files, analyze source code, identify security weaknesses, and generate intelligent recommendations.

# PRROBLEM STATEMENT

- Many developers deploy applications without performing comprehensive security analysis.
- Existing manual code review processes require significant time and experienced security professionals.
- Security vulnerabilities such as hardcoded credentials, SQL Injection risks, weak configurations, and outdated dependencies often remain undetected.
- Traditional vulnerability assessment tools are complex and may not provide developer-friendly recommendations.
- There is a need for an intelligent automated platform that can quickly detect vulnerabilities and provide actionable security improvements before deployment.

ENGINEER TO EXCEL

# OBJECTIVE

- To develop an AI-powered secure code review platform that automatically detects vulnerabilities and improves application security before deployment.
- Upload and validate project files securely before analysis.
- Automatically scan source code and identify security vulnerabilities using rule-based and AI-assisted techniques.
- Analyze detected vulnerabilities, calculate project security score, and classify project risk levels.
- Generate detailed security reports and provide recommendations to help developers fix identified issues.

# LITERATURE SURVEY

| No. | Year | Literature / Study | Main Focus | Key Findings | Relevance to Your Project |
|---|---|---|---|---|---|
| **1** | **2024** | **Integrating Machine Learning into CI/CD Pipelines** | AI/ML in DevSecOps | The study examines how AI and ML can be integrated into CI/CD to improve security testing, vulnerability detection, automated remediation and threat intelligence. (IJISAE) | Supports your idea of integrating **automated vulnerability detection into the software development workflow**. |
| **2** | **2025** | **DevSecOps Practices and Tools** | DevSecOps practices and security tools | A literature review of **103 publications** identified **39 DevSecOps practices**, covering security automation and integration throughout software development. (Springer) | Provides the theoretical foundation for your **DevSecOps-based security scanning platform**. |
| **3** | **2025** | **AI-Based Software Vulnerability Detection: A Systematic Literature Review** | AI-based vulnerability detection | The review analyzes AI-based software vulnerability detection approaches and highlights issues such as **dataset quality, reproducibility and interpretability**. (arXiv) | Supports your future enhancement of DevDeployShield with **AI-based vulnerability detection** instead of relying only on rule-based scanning. |
| **4** | **2025** | **Comparative Analysis of AI-Driven Security Approaches in DevSecOps** | AI/ML security automation | The study compares AI-driven DevSecOps approaches and identifies challenges involving **empirical validation, scalability and integration of AI into security automation**. (arXiv) | Helps justify your system's combination of **automated scanning, risk analysis and security dashboards**. |
| **5** | **2025** | **Static Analysis Techniques for Secure Software: A Systematic Review** | Static application security testing | The research emphasizes timely vulnerability detection in DevSecOps and examines how static-analysis and AI-based techniques can improve software security. (ResearchGate) | Directly relates to your **source-code scanners** for passwords, HTTP, SQL injection, debugging statements and authentication weaknesses. |
| **6** | **2025** | **Static Security Vulnerability Scanning of Proprietary and Open-Source Software** | Automated source-code vulnerability scanning | The study presents an end-to-end process for routinized code scanning, vulnerability prioritization and remediation within the SDLC, while discussing automation and AI as future enhancements. (arXiv) | Closely matches your architecture: **scan project → detect vulnerabilities → classify severity → generate recommendations**. |
| **7** | **2026** | **LLM-Driven DevSecOps for Secure Software Engineering** | LLM + DevSecOps | The proposed framework combines **SAST, retrieval-augmented generation, automated vulnerability remediation, human approval and post-remediation validation**. (IJAIDSML) | Provides a strong future direction for DevDeployShield: **AI-assisted explanation and automatic remediation of detected vulnerabilities**. |

ENGINEER TO EXCEL

# Existing System vs Proposed System

| Existing System | Proposed System |
| --- | --- |
| Manual code review process | Automated AI-powered vulnerability scanning |
| Time-consuming security analysis | Fast and intelligent project scanning |
| Requires security experts | Easy for developers to use |
| Limited vulnerability identification | Automatically detects multiple security vulnerabilities |
| Minimal security guidance | Provides security recommendations, risk level, and security score |

ENGINEER TO EXCEL

# Tools & Technologies Used

- Java 21 – Core programming language for backend implementation.
- Spring Boot – Web framework for building the DevDeployShield application.
- Thymeleaf + HTML + CSS + JavaScript – Frontend development and user interface.
- Maven – Dependency management and project build automation.
- ZIP Utilities & Java File I/O – Project upload, extraction, and source code scanning.

# Module 1- Project Upload and Secure File Management Description

This module provides a secure interface for uploading Java or Spring Boot projects in ZIP format. It validates the uploaded file, stores it safely on the server, extracts the project contents, and prepares the project for further security analysis.

## Functions

ZIP project validationUpload statusZIP extractionProjec t storageProject initialization

**Input** Java/Spring Boot ZIP Project

**Output** Extracted project ready for scanning.

ENGINEER TO EXCEL

# Module 2 AI-Powered Vulnerability Detection Engine Description

- This module performs static source code analysis by scanning every file inside the uploaded project. It detects security vulnerabilities using predefined security rules and pattern matching techniques.

## Functions

Recursive file scanning

Pattern matching

Security rule

Vulnerability identification

Severity classification

**Output** List of detected vulnerabilities.

## How It Works

Source Code Upload

↓

AI Scans & Analyzes the Code

↓

Detects Security Vulnerabilities

↓

Generates Report with Risk Level & Fix Suggestions

# Module 3 Secure Code Review and Risk Assessment Description

- This module evaluates the vulnerabilities detected during scanning and determines the overall security posture of the uploaded project.

## Functions

Count vulnerabilities

Calculate security score

Risk level determination

Project statistics

Security recommendations

AI recommendation

**Output** **Security Score**

**Work flow**

Source Code Review

↓

Identify Security Risks

↓

Assign Risk Level (Low/Medium/High/Critical)

↓

Provide Secure Coding Recommendations

# Module 4 Security Dashboard and Report Generation Description

- This module presents the scan results in a user-friendly dashboard and generates downloadable security reports for developers.

## Functions

Interactive dashboard

Vulnerability table

Security score visualization

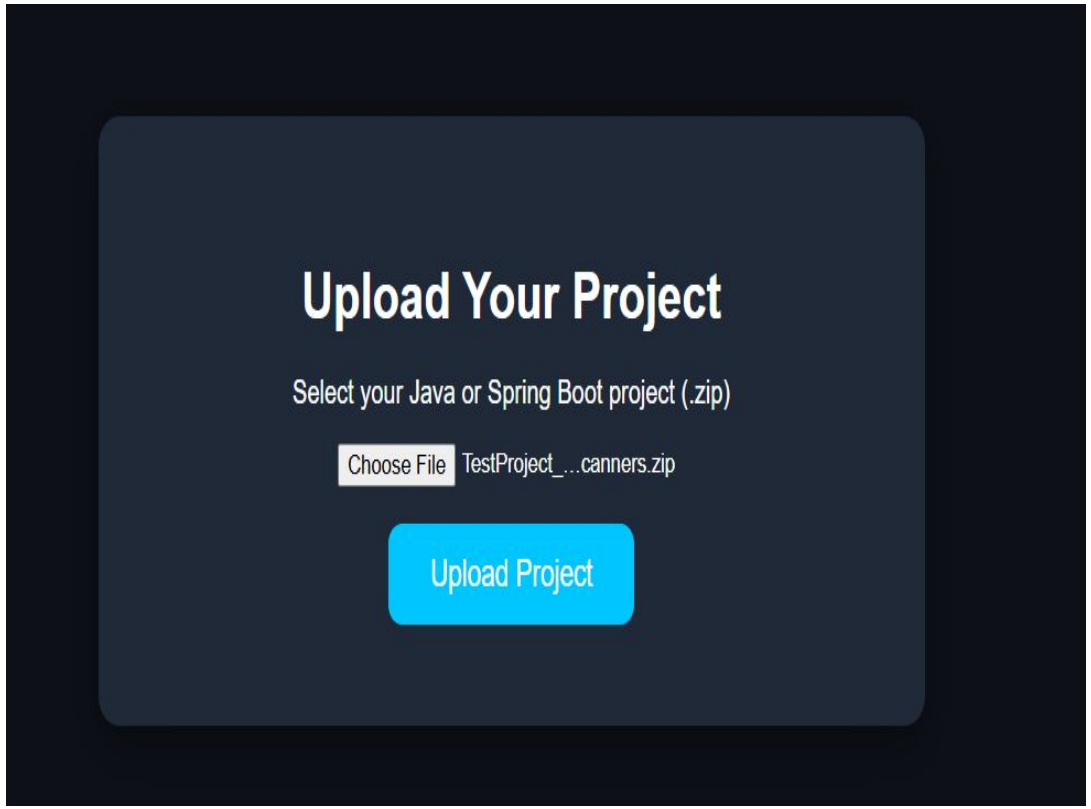
Risk visualization

PDF report generation

## Output

- Security Dashboard
- PDF Report
- Scan Summary

# output


Upload Your Project
Select your Java or Spring Boot project (.zip)
Choose File
TestProject_...canners.zip
Upload Project


DevDeployShield
AI Security Scan Result
Dashboard
AI Scanner
Upload Project
Risk Analysis
Security Dashboard
Report Generator
Settings
Scan Completed Successfully
Your uploaded project has been analyzed by DevDeployShield.
Security Score
57%
Overall Security Rating
High Risk
9
High severity findings
Medium Risk
2
Medium severity findings
Low Risk
4
Low severity findings
Project Information
Item
Value
Project Name
TestProject_All_8_Scanners
Total Files
3
Java Files
1
pom.xml Files
1
Properties Files
1
Total Vulnerabilities
15
Risk Level
HIGH

## Detected Vulnerabilities

Security issues identified during source-code analysis.

15 Findings

HIGH **Hardcoded Password**

App.java Line 11

Hardcoded credentials detected in source code. Storing passwords directly in the application source code can expose sensitive credentials.

Detected Code

```
String password = "admin123";
```

**Recommendation**

Move the password to an environment variable or a secure secret management system.

HIGH **Hardcoded Password**

App.java Line 14

Hardcoded credentials detected in source code. Storing passwords directly in the application source code can expose sensitive credentials.

Detected Code

```
String apiKey = "ABC123SECRETKEY";
```

**Recommendation**

Move the password to an environment variable or a secure secret management system.

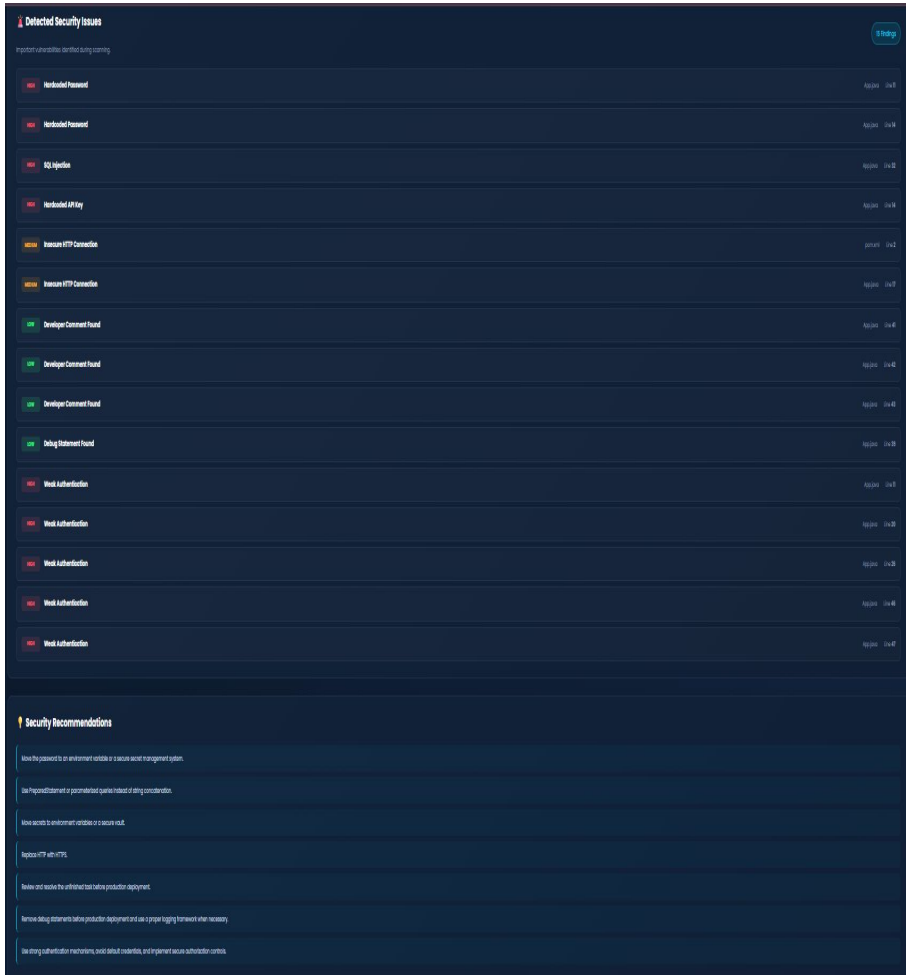
## AI Security Recommendations

- ✓ Move the password to an environment variable or a secure secret management system.
- ✓ Use PreparedStatement or parameterized queries instead of string concatenation.
- ✓ Move secrets to environment variables or a secure vault.
- ✓ Replace HTTP with HTTPS.
- ✓ Review and resolve the unfinished task before production deployment.
- ✓ Remove debug statements before production deployment and use a proper logging framework when necessary.
- ✓ Use strong authentication mechanisms, avoid default credentials, and implement secure authorization controls.

Scan Another Project

Print Security Report


DevDeployShield
Dashboard
AI Scanner
Upload Project
Risk Analysis
Security Dashboard
Report Generator
Settings
Risk Analysis
Security risk assessment of the scanned project
PROJECT
TestProject_All_8_Scanners
SECURITY SCORE
57%
Overall security health
RISK LEVEL
HIGH
Current project risk
TOTAL FINDINGS
15
Security issues detected
RISK PERCENTAGE
43%
Estimated project risk
Security Assessment
The project has significant security risks. High and medium severity findings require attention.
Vulnerability Severity
Distribution of detected security issues
High Risk 9
Medium Risk 2
Low Risk 4
Risk Distribution
Findings by severity
HIGH 9
MEDIUM 2
LOW 4
Vulnerability Types
Most frequently detected issues
Hardcoded Password 2
SQL Injection 1
Hardcoded API Key 1
Insecure HTTP Connection 2
Developer Comment Found 3
Debug Statement Found 1
Weak Authentication 5
Project Risk Information
Security-related project statistics
Total Files 3
Java Files 1
pom.xml Files 1
Properties Files 1
View Scan Results
Scan Another Project


Detected Security Issues
Important vulnerabilities identified during scanning
15 findings
HIGH Hardcoded Password App.java Line 9
HIGH Hardcoded Password App.java Line 14
HIGH SQL Injection App.java Line 22
HIGH Hardcoded API Key App.java Line 16
MEDIUM Insecure HTTP Connection pom.xml Line 2
MEDIUM Insecure HTTP Connection App.java Line 17
LOW Developer Comment Found App.java Line 41
LOW Developer Comment Found App.java Line 42
LOW Developer Comment Found App.java Line 43
LOW Debug Statement Found App.java Line 28
HIGH Weak Authentication App.java Line 9
HIGH Weak Authentication App.java Line 28
HIGH Weak Authentication App.java Line 29
HIGH Weak Authentication App.java Line 46
HIGH Weak Authentication App.java Line 47
Security Recommendations
Move the password to an environment variable or a secure secret management system.
Use PreparedStatement or parameterized queries instead of string concatenation.
Move secrets to environment variables or a secure vault.
Replace HTTP with HTTPS.
Review and resolve the unfinished task before production deployment.
Remove debug statements before production deployment and use a proper logging framework when necessary.
Use strong authentication mechanisms, avoid default credentials, and implement secure authentication controls.

ENGINEER TO EXCEL

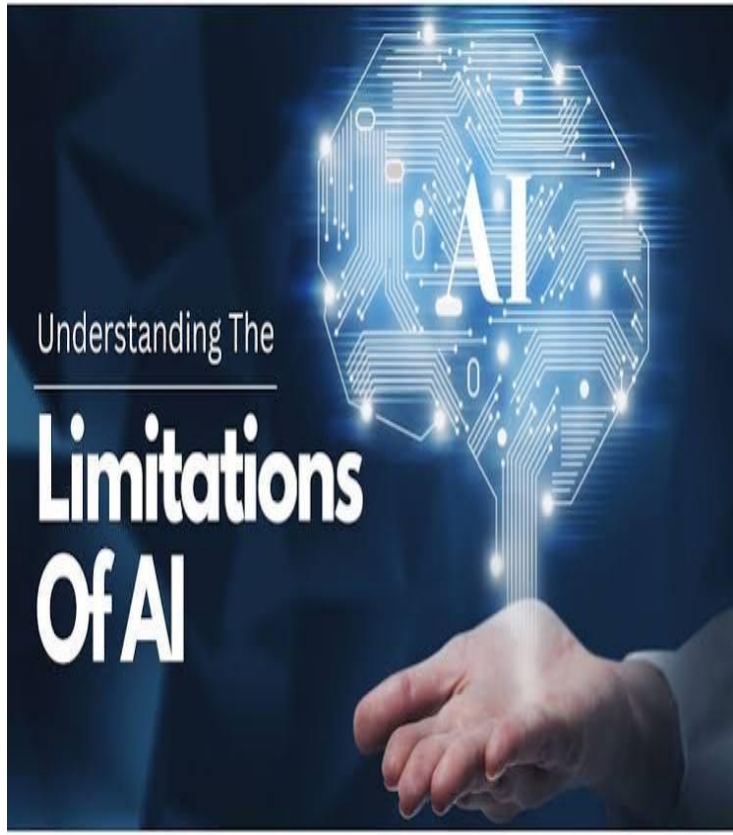
# ADVANTAGES

- Automates vulnerability detection and secure code review
- Reduces manual effort and improves development efficiency.
- Provides developer-friendly security recommendations.
- Improves application security before deployment.
- Supports DevSecOps practices by integrating security into development.

ENGINEER TO EXCEL

# LIMITATIONS

- Currently supports only Java and Spring Boot projects.
- Uses rule-based detection for many vulnerabilities in current version.
- Does not perform dynamic runtime security testing.
- Large projects may require additional scanning time.
- Advanced AI models require more computational resources for deeper analysis.



ENGINEER TO EXCEL

# CONCLUSION

- DevDeployShield simplifies secure software development by automating vulnerability detection.
- The platform improves application security through intelligent code analysis and secure coding recommendations.
- Developers can identify and fix security issues before software deployment.

  The project demonstrates practical implementation concepts using Spring Boot.
- Future enhancements will include machine learning–based vulnerability prediction, real-time monitoring, cloud deployment, and support for multiple programming languages.

ENGINEER TO EXCEL

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Deemed to be University)
2005
Knowledge Conquers All

# THANK YOU